

TaxPal: Personal Finance & Tax Estimator for Freelancers

Description

TaxPal is a full-stack financial management platform tailored to freelancers and independent contractors. The system centralizes income and expense tracking, budgeting, reporting, and quarterly tax planning so users can monitor cash flow, stay compliant with filing deadlines, and export well-formatted financial summaries. The application consists of a React single-page app backed by an Express API that persists data in MongoDB and issues JWTs for authenticated access.

Project Objectives

- Give users a consolidated ledger for income and expense activity.
- Surface real-time financial metrics through dashboards, charts, and summaries.
- Provide monthly budgeting tools with progress indicators and health scores.
- Generate downloadable financial reports in PDF, CSV, and XLSX formats.
- Capture quarterly tax estimates and reminders to reduce last-minute surprises.
- Deliver secure account lifecycle management with OTP-based password recovery.

Core Features

- Secure authentication (signup, login, change password, delete account) with bcrypt hashing and JWT sessions.
- Two-step password reset flow that emails one-time codes and verifies before issuing reset tokens.
- CRUD operations for income and expense transactions with filters, sorting, pagination, and dashboard summaries.
- User-defined categories for income and expense classification with color tagging.
- Monthly budgets tied to expense categories, including utilization charts and health indicators.
- Automated financial reports (Income Statement, Balance Sheet, Cash Flow, Expense Summary) with export helpers.
- Tax estimator that models quarterly liabilities, stores historical runs, and seeds reminders in a tax calendar.
- Tax calendar of personalized reminders persisted in local storage with delete actions.
- Settings area for profile metadata, category management, notification preferences placeholder, and security operations.
- Toast notifications, modals, and responsive layouts to streamline user interactions.

Technology Stack

- **Frontend:** React 19 (Create React App), React Router v7, Recharts, Framer Motion, jsPDF, jsPDF-AutoTable, XLSX, custom modal and toast providers.

- **Backend:** Node.js 18+, Express 4, Mongoose 7, JSON Web Tokens, bcryptjs, express-validator, Nodemailer, Morgan, CORS middleware.
- **Database:** MongoDB (Atlas or self-managed) accessed via Mongoose models.
- **Tooling:** npm for dependency management, nodemon for backend hot reloads.

System Architecture

- **Client SPA:** React renders authenticated routes behind a layout shell, manages local storage, and communicates exclusively through the services/api.js module. Toasts, modals, and router transitions are handled client-side for a smooth UX.
- **API Layer:** Express hosts versioned REST endpoints under /api/, enforcing JSON payloads, JWT authentication, and validation guards. Controllers coordinate business logic by delegating to Mongoose models and shared utilities.
- **Persistence Layer:** MongoDB stores normalized collections (users, transactions, categories, budgets, reports, taxEstimates). Aggregations power dashboards, reports, and summaries without additional OLAP tooling.
- **Email Service:** Nodemailer abstracts SMTP transport and is invoked for OTP delivery. Credentials and from-address are owned via environment variables.
- **Security:** JWT bearer tokens protect all financial endpoints; rate limiting can be added at the router level if traffic grows. Passwords never leave the database unhashed.
- **Hosting Considerations:** The backend can be deployed as a Node process behind a reverse proxy (NGINX/Cloudflare). The frontend builds to static assets served by CDN or the same Node host when using a static middleware.

Repository Structure

Taxpal_team5-1/

```

├── backend/
│   ├── package.json
│   ├── src/
│   │   ├── config/      # Database connection helpers
│   │   ├── controllers/  # Route handlers (auth, transactions, budgets, reports, taxes, categories)
│   │   ├── middleware/   # JWT auth guard and request validation
│   │   ├── models/      # Mongoose schemas
│   │   ├── routes/      # Express routers
│   │   └── utils/        # Email and report builders, token signer
│   └── server.js
├── frontend/
└── package.json
  
```

```
| └─ public/
| └─ src/
|   └─ components/    # Feature modules and UI building blocks
|   └─ services/      # API abstraction for REST calls
|   └─ utils/         # Local-storage helpers, export mappers
|   └─ App.js         # Route configuration
└─ TAXPAL Project Documentation.docx.txt
```

Prerequisites

- Node.js 18 or later and npm.
- MongoDB 6.x/7.x instance (local or Atlas cluster) with network access for the backend.
- SMTP credentials compatible with Nodemailer (Gmail app password, transactional email service, etc.).
- Recommended tooling: VS Code or any modern JavaScript IDE.

Initial Setup

1. Clone the repository:

```
git clone https://github.com/springboardmentor0316/Taxpal_team5.git
cd Taxpal_team5
```

2. Install backend dependencies:

```
cd backend
npm install
```

3. Configure backend environment variables by copying .env and updating the values (see Backend Environment Variables).

4. Install frontend dependencies:

```
cd ../frontend
npm install
```

5. Create a frontend/.env file with the API base URL (see Frontend Environment Variables).

Sample Data & Demo Accounts

- No seed script ships with the repo. To showcase features quickly:
- Create a demo user via the sign-up flow (e.g., demo@taxpal.dev with strong password).

- Add at least one income and expense category in Settings → Categories.
- Record several income/expense transactions covering multiple months to populate dashboard charts.
- Create a budget for an expense category to visualize health indicators.
- Generate a report and export it to confirm export utilities work end-to-end.
- For automated demos, consider writing a one-off script using Mongoose models to insert sample data, ensuring you do not commit credentials to source control.

Running the Application

1. Start the backend API

```
cd backend
```

```
npm run dev      # uses nodemon; use npm start for production mode
```

- API base URL (default): `http://localhost:5001/api`
- Health check: `GET http://localhost:5001/api/health`

2. Start the frontend SPA

```
cd frontend
```

```
npm start
```

- Served at `http://localhost:3000`
- The React app reads `REACT_APP_API_URL` to reach the backend.

Backend Environment Variables

- Set these in `backend/.env` (values shown are examples only):
- `PORT=5001` — Express server port.
- `MONGODB_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/taxpal` — MongoDB connection string.
- `MONGODB_DNS_SERVERS=1.1.1.1,8.8.8.8` — Optional fallback DNS servers for SRV lookups.
- `JWT_SECRET=<random-long-string>` — Secret for signing JWT access tokens.
- `CLIENT_URL=http://localhost:3000,http://localhost:3001` — Comma-separated list of allowed origins for CORS.
- `NODE_ENV=development` — Standard Node environment flag.
- Email configuration (choose service or host-based):
- `EMAIL_SERVICE=gmail`
- or `EMAIL_HOST=smtp.yourprovider.com`
- `EMAIL_PORT=465`
- `EMAIL_SECURE=true`
- `EMAIL_USER=<smtp-user>`
- `EMAIL_PASS=<smtp-password-or-app-pass>`

- EMAIL_FROM="TaxPal <no-reply@taxpal.com>" — From header shown in OTP emails.

Frontend Environment Variables

- Create frontend/.env if it does not exist:
- REACT_APP_API_URL=http://localhost:5001/api
- You can point this to a production API host when deploying.

Authentication & Security

- Passwords are hashed with bcrypt (10 rounds) before storage.
- Signed JWTs contain the user ID and expire after seven days.
- Protected API routes use auth middleware to validate bearer tokens and attach req.user.
- Password reset uses a six-digit OTP hashed and stored temporarily, followed by a single-use reset token.
- Account deletion removes all related documents (transactions, budgets, categories, reports, and tax estimates).
- Tokens are stored client-side in localStorage; future hardening can include rotating refresh tokens or moving to HttpOnly cookies.
- Request body validation prevents malformed payloads from reaching business logic; extend validators whenever new fields are introduced.
- CORS whitelist is derived from CLIENT_URL; keep production URLs updated to avoid exposing the API to unknown origins.
- Consider enforcing HTTPS everywhere and enabling HSTS at the proxy layer to prevent downgrade attacks.

Security Hardening Roadmap

- Add MFA (email or authenticator app) for critical actions such as deleting accounts or exporting sensitive reports.
- Implement rate limiting/throttling on auth routes to deter brute-force attempts (libraries: express-rate-limit, redis rate store).
- Introduce audit logging for authentication events, password resets, and destructive actions (budgets/reports deletions).
- Periodically rotate JWT secrets and SMTP credentials; document rotation procedures and required redeloys.
- Evaluate migrating token storage to HttpOnly secure cookies to mitigate XSS token theft.
- Consider enabling SSO/OAuth for enterprise clients (Google/Microsoft sign-in) while maintaining local accounts as fallback.

Database Models

User

- **_id**: ObjectId primary key.
- **username**: Unique, trimmed string displayed across the app.
- **fullName**: Optional full display name.
- **email**: Unique, lowercase string used for login and OTP delivery.
- **password**: Bcrypt hash (salted in pre('save') hook).
- **country, incomeBracket**: Profile metadata consumed by dashboards and estimator defaults.
- **resetCodeHash, resetCodeExpires**: Temporary OTP verification storage.
- **resetPasswordToken, resetPasswordExpires**: Single-use token for password reset step two.
- **createdAt, updatedAt**: Managed by Mongoose timestamps.

Transaction

- **user**: Owner reference (ObjectId).
- **type**: income or expense.
- **category**: Free-form string keyed against user categories.
- **description**: Required headline shown in tables and exports.
- **notes**: Optional supplemental text.
- **amount**: Positive Number stored in the currency unit selected by the user (currently INR-centric formatting on the client).
- **date**: Date object; indexes support chronological queries.
- **createdAt, updatedAt**: Auditing metadata.

Category

- **user**: Owner (nullable for defaults).
- **name**: Unique per user/type combination.
- **type**: income or expense.
- **color**: Hex string used for chips and charts.
- **isDefault**: Boolean flag to prevent duplicate defaults.

Budget

- **user**: Owner.
- **category**: Expense category key (validated on create/update).
- **limit**: Monthly ceiling.
- **month**: YYYY-MM identifier used for reporting and dedupe.
- **note**: Optional description.

Report

- **user:** Owner.
- **name:** Human-friendly label (defaults to \${type} - \${period}).
- **period, periodKey:** Rendered label and canonical identifier.
- **startDate, endDate:** Range boundaries used in queries.
- **reportType:** Enum stored as free string (Income Statement, Balance Sheet, Cash Flow, Expense Summary).
- **format:** Requested export format (PDF, CSV, XLSX).
- **filePath:** Optional pointer when storing generated assets externally.
- **payload:** Serialized summary used for deterministic exports.

TaxEstimate

- **user:** Owner.
- **year, quarter:** Composite unique key (unique index).
- **country, state, filingStatus:** Inputs for estimation logic.
- **grossIncome, businessExpenses, healthInsurancePremiums, retirementContribution, homeOfficeDeduction:** Monetary inputs.
- **estimatedTax, effectiveRate:** Calculated outputs kept for history charts.
- **dueDate:** Auto-derived from quarter.
- **notes:** Additional annotations.

Backend Components

Controllers

- **authController:** Handles signup/login, password reset flow, password change, and account deletion while coordinating OTP delivery.
- **transactionController:** CRUD operations, filtering, pagination, and summary aggregation for dashboard charts.
- **budgetController:** Validates categories, enforces monthly uniqueness, and aggregates expenses to compute health metrics.
- **categoryController:** Manages user-defined categories, ensuring uniqueness and providing clean payloads to clients.
- **reportController:** Resolves time ranges, builds report payloads, and stores metadata for export.
- **taxEstimateController:** Upserts quarterly tax estimates, normalizes numbers, and resolves due dates based on quarter.

Middleware

- **auth:** Validates JWT, attaches authenticated user ID, and rejects unauthorized requests before hitting controllers.
- **validateRequest:** Consolidates express-validator errors and sends structured responses to clients.

Utilities

- **generateToken.js:** Signs JWTs with configured expiry.

- **email.js:** Lazily instantiates Nodemailer transporter using env-configured SMTP settings.
- **reportBuilder.js:** Encapsulates aggregation pipelines and formatting helpers for reports.
- **taxCalendarStore.js** (frontend): Implements local storage persistence with a pub/sub API for tax reminders.

API Surface

All endpoints are prefixed with /api. Routes marked with 🔒 require a valid Authorization: Bearer <JWT> header.

Auth (/api/auth)

- POST /signup — Register user, enforce password policy.
- POST /login — Issue JWT with basic profile payload.
- POST /forgot-password — Send six-digit OTP email.
- POST /verify-reset-code — Validate OTP and return reset token.
- POST /reset-password — Reset password using reset token.
- POST /change-password 🔒 — Update password after verifying current password.
- DELETE /account 🔒 — Permanently delete user and related data.

Transactions (/api/transactions) 🔒

- POST / — Create income or expense.
- GET / — List transactions with pagination, filtering, search, and sorting.
- GET /:id — Fetch single transaction.
- PUT /:id — Update transaction fields.
- DELETE /:id — Remove transaction.
- GET /summary — Aggregated metrics powering the dashboard (totals, monthly series, category breakdown, daily/quarterly trends).

Budgets (/api/budgets) 🔒

- GET / — List budgets (optionally filtered by month), with spending summary.
- POST / — Create month/category budget; validates category existence.
- PUT /:id — Update limit, note, or category.
- DELETE /:id — Remove a budget entry.

Categories (/api/categories) 🔒

- GET / — Return income and expense categories for the user.
- POST / — Add custom category with optional color.

- PUT /:id — Rename or recolor category.
- DELETE /:id — Delete category.

Reports (/api/reports) 🔒

- GET / — List stored report payloads with filters (period, type, format, search, date range).
- POST / — Generate and persist a report; resolves period ranges, aggregates transactions, stores payload for exports.
- DELETE /:id — Delete report snapshot.

Tax Estimates (/api/tax-estimates) 🔒

- GET / — List saved quarterly estimates sorted by year and quarter.
- POST / — Create or update (upsert) estimate with derived due date.
- DELETE /:id — Remove saved estimate.

Domain Workflows

User Onboarding

1. Visitor selects **Sign Up** and submits username, email, password, confirmation, country, and income bracket.
2. Backend hashes credentials, persists the user, and returns a JWT plus profile payload.
3. Frontend stores taxpal_token and taxpal_user, then routes to the dashboard.
4. Categories screen can be visited immediately to seed custom categories.

Password Recovery

1. From the Sign In page, user chooses **Forgot Password** and provides email.
2. Backend sends OTP email and stores hashed code with 15-minute expiry.
3. User enters code; API validates and issues one-time reset token.
4. Reset Password page consumes token, posts new password, and navigates back to Sign In.
5. Token is invalidated server-side to prevent reuse.

Recording Financial Activity

1. From dashboard or transactions table, user opens **Record Income/Expense** modal.
2. Client POSTs to /transactions including type, category, amount, date, description, notes.

3. On success, the table refreshes and dashboard summary is re-fetched to include the new totals.

Budget Monitoring

1. Budgets screen fetches current month budgets plus expense totals aggregated server-side.
2. User creates a new budget selecting an expense category and monthly limit.
3. Transactions automatically roll up into spent values, updating progress bars and health chips.
4. Budgets may be edited or deleted; updates are persisted via PUT/DELETE endpoints.

Report Generation & Export

1. User chooses report type, period preset (or custom range), format, and optional notes.
2. Backend resolves date ranges, aggregates transactions, stores the payload, and returns report metadata.
3. Client lists saved reports, enabling PDF/CSV/XLSX exports using the stored payload for deterministic output.
4. Reports can be removed when no longer needed, freeing storage.

Tax Planning

1. Estimator form captures quarter, year, jurisdiction, filing status, income, and deductions.
2. Client calculates a preview instantly; saving posts to /tax-estimates for persistence.
3. History view renders saved estimates and cost bars grouped by quarter.
4. **Add Reminder** seeds tax calendar events, which the separate calendar view manages (local storage persistence).

API Usage Examples

Login

bash

```
curl -X POST http://localhost:5001/api/auth/login \  
-H "Content-Type: application/json" \  
-d '{"email":"freelancer@example.com","password":"Password1!"}'
```

Response:

json

```
{
```

```
"message": "Login successful",
"token": "<jwt>",
"user": {
  "id": "65f6a3e7...",
  "username": "freelancer01",
  "fullName": "Ava Patel",
  "email": "freelancer@example.com",
  "country": "India",
  "incomeBracket": "₹6,00,000 - ₹10,00,000"
}
```

Create Transaction

```
bash
curl -X POST http://localhost:5001/api/transactions \
-H "Authorization: Bearer <jwt>" \
-H "Content-Type: application/json" \
-d '{
  "type": "expense",
  "category": "Software",
  "amount": 4500,
  "date": "2025-03-12T00:00:00.000Z",
  "description": "Project management subscription",
  "notes": "Annual renewal"
}'
```

Fetch Transaction Summary

```
bash
curl -X GET http://localhost:5001/api/transactions/summary \
-H "Authorization: Bearer <jwt>"
```

Response excerpt:

json

```
{
  "metrics": {
    "totalIncome": 520000,
    "totalExpense": 310000,
    "netIncome": 210000,
    "lastUpdated": "2025-03-30T07:52:11.234Z"
  },
  "recentTransactions": [
    { "description": "Retainer invoice", "amount": 45000, "type": "income", "date": "2025-03-29T00:00:00.000Z" }
  ],
  "monthlySeries": [
    { "month": "2025-01", "type": "income", "total": 180000 }
  ]
}
```

Save Tax Estimate

bash

```
curl -X POST http://localhost:5001/api/tax-estimates \
-H "Authorization: Bearer <jwt>" \
-H "Content-Type: application/json" \
-d '{
  "quarter": "Q2",
  "year": 2025,
  "country": "India",
  "state": "Karnataka",
  "filingStatus": "Single",
  "grossIncome": 120000,
  "businessExpenses": 35000,
```

```
"healthInsurancePremiums":6000,  
"retirementContribution":10000,  
"homeOfficeDeduction":5000,  
"estimatedTax":9000,  
"effectiveRate":0.21,  
"notes":"Includes June retainer"  
}'
```

Generate Report

```
bash  
curl -X POST http://localhost:5001/api/reports \  
-H "Authorization: Bearer <jwt>" \  
-H "Content-Type: application/json" \  
-d '{  
  "reportType":"Income Statement",  
  "format":"PDF",  
  "periodKey":"current-month",  
  "notes":"Monthly snapshot"  
}'
```

Frontend Modules

- **Routing** (App.js): Public routes for signin/signup/reset and a protected layout that wraps dashboard, budgets, tax estimator, tax calendar, reports, transactions, and settings screens.
- **Authentication:** Signup enforces password complexity, collects profile metadata (country, income bracket). Signin stores JWT (taxpal_token) and profile (taxpal_user) in local storage. Forgot/reset password implements OTP verification and session storage of temporary reset tokens.
- **Dashboard:** Fetches /transactions/summary to render cards, bar charts, pie charts, and recent activity. Supports quick access to record income/expense via modal dialogs.
- **Transactions:** Table with filters (type, category, search, date range), pagination controls, and inline totals. Uses Income/Expense modals for CRUD.
- **Budgets:** Visualizes budget vs actual per category, shows utilization chips (Good/Warning/Bad), allows creating, editing, and deleting budgets tied to expense categories.

- **Reports:** Wizard for choosing report type, period, format, and notes. Persists snapshots, supports filtering list, and offers exports to PDF/CSV/XLSX via helpers in `utils/reportExport.js`.
- **Tax Estimator:** Calculates quarterly taxes based on country/state, filing status, deductions, and displays result charts. Saves history to backend and seeds reminders into the local tax calendar store.
- **Tax Calendar:** Lists reminders stored in local storage (`taxpal_tax_calendar_events_v1`) with delete actions; integrates with estimator workflow.

Settings:

- **Profile:** Edit username and country locally, view email (read-only).
- **Categories:** CRUD interface that syncs with backend and notifies other screens through custom events.
- **Notifications:** Placeholder for future messaging preferences.
- **Security:** Sign out all sessions (clears token), change password modal, and account deletion flow with confirmation dialogs.

Shared Systems: ToastProvider for feedback banners, ModalProvider for confirmations and alerts, custom hooks for local storage synchronization.

Frontend Component Reference

- `components/dashboard/`: Dashboard page, summary cards, charts (recharts) with dynamic range selectors.
- `components/transactions/`: Table, filters, pagination controls, and modals that wrap the shared income/expense forms.
- `components/income/` & `components/expense/`: Modal forms shared between dashboard and transactions contexts.
- `components/budgets/`: Budget list, chart, form, and helper utilities for status chips.
- `components/report/`: Report builder wizard, history list, and integration hooks for export utilities.
- `components/taxEstimator/`: Form logic, Recharts bar visualization, history list, and reminder modal seed.
- `components/taxCalendar/`: Calendar list view and store subscription logic.
- `components/settings/`: Nested routes for profile, categories, notifications, security, and reusable subcomponents (e.g., delete account modal).
- `components/sidebar/` & `components/layout/`: Navigation shell, responsive sidebar, top header.
- `components/common/ProtectedRoute`: Guards private routes by checking stored token and user payload.
- `components/modal/` & `components/toast/`: Context providers, hooks, and UI primitives used globally.
- `services/api.js`: Fetch wrapper handling headers, auth injection, query serialization, error normalization.

- `utils/`: Local storage helpers (`user.js`), report payload formatters, tax calendar store, and generic helpers.

Data Persistence & State

- JWT token resides in `localStorage` under `taxpal_token`; profile info under `taxpal_user`.
- Tax calendar reminders persist locally, enabling offline retention without backend writes.
- Reset password temporary tokens are stored in `sessionStorage` and cleared after use.
- Backend enforces model-level uniqueness and uses MongoDB aggregation pipelines for summaries.
- Custom storage events (`taxpal:user-updated`, `taxpal:categories-updated`) keep independent components synchronized without prop drilling.
- Modals and toasts rely on React Context providers so deep children can trigger global UI affordances.
- API errors bubble through the `useToast` hook, giving consistent messaging for validation and server failures.

Error Handling & Logging

- Backend wraps controller logic in `try/catch` blocks and funnels unknown issues to a shared `handleServerError`, returning generic messages while logging stack traces to `stdout`.
- Validation failures use `express-validator` with a middleware that returns 422 responses containing normalized error payloads.
- Authentication middleware intercepts missing/invalid tokens and short-circuits with 401 responses to protect downstream controllers.
- Morgan (dev format) is enabled in non-production environments for concise request logging.
- Frontend surfaces server errors via toast notifications and inline form validation messages; destructive actions additionally prompt user confirmation via modals.

Monitoring & Metrics (Recommended)

- Attach structured logging (e.g., Winston + JSON logs) before production to feed centralized log management (Datadog, Logtail).
- Record API latency and error rates; lightweight options include Express middleware that captures duration and status.
- Capture frontend telemetry through browser analytics (e.g., Plausible, PostHog) focusing on conversion funnels for signup and report downloads.
- Enable uptime monitoring for both frontend and backend endpoints; health route (`/api/health`) can be used by external monitors.

Performance Considerations

- **Backend:** Optimize MongoDB aggregations with appropriate indexes (user, date, category). Avoid unbounded projections; paginate large result sets.
- **Frontend:** Memoize expensive calculations using React hooks (useMemo, useCallback) as already applied in dashboard/transactions to prevent unnecessary re-renders.
- **Network:** Batch API calls where possible (dashboard uses a single summary endpoint). Keep payload sizes small by excluding unused fields.
- **Build size:** CRA already code-splits vendor chunks; monitor bundle analyzer reports if dependencies grow (e.g., consider dynamic imports for heavy modules).
- **Caching:** Introduce HTTP caching headers or client-side caching for static resources and non-mutating endpoints to reduce load under scale.

Email & OTP Flow

1. User submits forgot-password form with email.
2. Backend generates 6-digit code, hashes it, stores expiry (15 minutes), and emails via Nodemailer.
3. User enters code; backend verifies and returns a signed reset token.
4. User submits new password with reset token; backend saves hashed password and invalidates tokens.

Reporting & Exports

- reportBuilder.js aggregates transactions by period, category, and growth deltas, including previous-period comparisons.
- Frontend export utilities format payloads for jsPDF (with AutoTable for tabular sections), CSV (comma-separated), and XLSX (via SheetJS).
- Generated reports retain their payloads in MongoDB so exports are deterministic and repeatable.

UI & Accessibility Guidelines

- Typography, spacing, and color styles live in App.css and component-level stylesheets; reuse CSS utility classes (set-head, panel, btn, chip) to maintain visual consistency.
- Interactive elements include keyboard focus outlines and ARIA labels (e.g., buttons with icons specify aria-label).
- Forms use clear validation states and inline error messaging; ensure new components mirror this approach.
- Avoid hardcoding currency/locale strings; rely on Intl.NumberFormat / Intl.DateTimeFormat used throughout the dashboard and estimator.
- Maintain sufficient color contrast in new themes; existing palette leverages blues/purples/greens with accessible contrast ratios.

Styling & Theming Notes

- CSS modules are not used; component-level .css files follow BEM-like class naming to prevent collisions.
- Responsive layouts rely on flexbox and CSS grid utilities defined in shared styles. Test breakpoints at 320px, 768px, and 1024px when introducing new screens.
- Animations are handled sparingly via Framer Motion to prevent layout shifts; keep transitions performant (<150ms) and disable heavy animations on low-powered devices if detected.

Testing & Quality Notes

- No automated tests are currently configured beyond the default CRA testing dependencies; manual verification is recommended.
- Key user journeys to cover manually: signup/login, password reset, transaction CRUD, budget edits, report generation/export, tax estimate save/delete.
- When adding tests, favor Jest and React Testing Library for the frontend and supertest/jest for backend APIs.
- Recommended backend test structure: backend/src/__tests__/** with isolated Mongo memory server (mongodb-memory-server) to run integration scenarios without touching production data.
- Frontend smoke tests should render each page component behind React Router's MemoryRouter and assert critical UI elements (forms, charts, tables) appear.
- Consider adding Cypress for end-to-end coverage of reporting exports and tax estimator flows where multiple forms interact.
- Linting/formatting: adopt ESLint + Prettier configurations consistent with CRA defaults before scaling contributions.

Troubleshooting

- **MongoDB connection errors:** Ensure MONGODB_URI is valid, IPs are whitelisted, and optional MONGODB_DNS_SERVERS values are reachable.
- **CORS denied:** Update CLIENT_URL to include the frontend origin(s).
- **OTP emails not sending:** Confirm SMTP credentials, provider restrictions (e.g., Gmail app passwords), and that EMAIL_FROM is set.
- **Frontend cannot reach API:** Verify REACT_APP_API_URL matches running backend and that both servers are on the same network.
- **Report generation failures:** Check that sufficient transaction data exists for the selected period and that custom date ranges are valid.

Deployment & DevOps

- **Frontend build:** Run `npm run build` in `frontend/` to produce optimized static assets inside `frontend/build`. Deploy to CDN (Netlify, Vercel, S3 + CloudFront) or serve via Express static middleware.
- **Backend production run:** Use `npm start` in `backend/` which executes `node src/server.js`. For resiliency, wrap with a process manager such as PM2 or systemd and set `NODE_ENV=production`.
- **Reverse proxy:** Terminate TLS at NGINX/Apache, proxy `/api/*` to the Node server, and serve static frontend assets from the build directory.
- **Environment segregation:** Maintain distinct `.env` files (or secret managers) per environment. Never commit secrets to version control; prefer Git ignored env files or platform secret stores.
- **Database migrations:** Schema changes require coordinated updates. Use MongoDB migration scripts (e.g., via `migrate-mongoose`) or custom scripts under a `scripts/` directory. Test on staging before production.
- **Backups & retention:** Schedule MongoDB exports (`mongodump` or Atlas snapshots) with at least daily frequency and store in encrypted object storage.
- **CI/CD:** Recommended pipeline steps—install dependencies, run lint/tests, build frontend, deploy backend, run smoke tests against staging, then promote to production.

Maintenance Guidelines

- Regularly back up MongoDB databases (snapshot Atlas cluster or run `mongodump`).
- Rotate JWT secrets and SMTP credentials if compromise is suspected.
- Monitor dependencies for security patches; update `package.json` and run regression checks.
- Use feature branches for new development, submit PRs, and protect the main branch for production-ready code.
- Document schema migrations and run them in staging before production adoption.

Contribution Workflow

- **Branching:** Follow a Git flow style—create feature/fix branches from main, open pull requests, and request review before merging.
- **Commit messages:** Use descriptive prefixes (`feat:`, `fix:`, `docs:`, `chore:`) to streamline changelog generation.
- **Code style:** Align with ESLint/Prettier defaults; avoid inline styles unless experimenting. Keep business logic in controllers/services rather than route files.
- **Reviews:** Validate all features locally, including login/logout flows, before requesting review. Provide screenshots or GIFs for UI changes.
- **Issue tracking:** Associate PRs with task IDs or issue numbers when using Jira/GitHub Issues for traceability.
- **Documentation:** Update this document, README, and inline comments whenever new capabilities or configuration steps are introduced.

Frequently Asked Questions

How do I invite a new user to the system?

Users self-register via the Signup page; admins are not required for account creation.

Where are financial exports stored?

Exports are generated on demand in the browser; report payloads are stored in MongoDB so you can re-export anytime.

Can I categorize transactions after importing them?

Yes. Edit a transaction from the Transactions screen and update its category. Budgets and reports will reflect the change automatically.

What happens if I delete my account?

The backend removes the user document and all related transactions, budgets, categories, reports, and tax estimates. Local reminders remain on the device until cleared manually.

Why does the reset code expire?

Verification codes are valid for 15 minutes for security. Request a new code if it lapses.

Future Enhancements

- Bank feed integrations for automated transaction imports.
- Shared workspaces for accountants or financial advisors.
- Multi-currency support and localized tax templates.
- Push/email notifications for upcoming tax deadlines and budget overspends.
- Deeper analytics with machine learning models to predict cash flow and tax liabilities.

Glossary

- **Budget Health:** Visual indicator summarizing how much of a monthly limit has been consumed (Good < 80%, Warning $\leq$ 100%, Bad > 100%).
- **Category:** User-defined label that groups transactions and budgets by purpose (e.g., Marketing, Consulting Income).
- **Effective Tax Rate:** Estimated annual tax divided by taxable income, expressed as a percentage.
- **OTP (One-Time Password):** Six-digit verification code emailed during password resets.

- **Report Payload:** JSON snapshot stored with each report that drives PDF/CSV/XLSX exports without re-querying the database.
- **Tax Calendar Event:** Reminder item persisted locally that highlights estimated payment due dates.
- **Transaction Summary:** Aggregated response from /transactions/summary that powers dashboard charts and cards.